

MPI 编程实验

2411567 陈涵

2026.06.07

目录

1	问题描述	2
1.1	期末研究报告宏观问题	2
1.2	本次多线程编程实验子问题	2
2	多线程算法设计与实现	2
2.1	MPI 算法思路	2
2.2	算法相关代码实现	3
2.2.1	MPI 初始化与 rank 分工	3
2.2.2	Master-Worker 任务分发	4
2.2.3	Worker 循环与停止信号	6
2.2.4	口令生成与 SIMD 哈希结合	7
3	实验及结果分析	7
3.1	实验结果	7
3.2	结果分析	8

1 问题描述

1.1 期末研究报告宏观问题

本课题旨在构建一套高效的混合并行口令猜测系统,需融合 MPI(跨节点任务分发)、OpenMP/Pthread 与 SIMD/GPU 等技术。核心研究问题在于:如何突破优先队列操作的锁竞争瓶颈、实现细粒度负载均衡、缓解 CPU-GPU/内存数据传输延迟,并量化分析“口令生成”与“哈希计算”在完整链路中的算力占比,从而设计最优的异构并行流水线。

1.2 本次多线程编程实验子问题

本次实验关注口令猜测算法在 MPI 多进程环境下的并行化实现。原始口令猜测程序以 PCFG 模型为基础,通过训练集统计口令结构和各类 segment 的出现频率,再按照概率从优先队列中不断取出 PT (Password Template),生成候选口令,并对候选口令进行 MD5 哈希计算。

在串行版本中,程序每次从优先队列中取出一个 PT,并遍历该 PT 最后一个 segment 的所有可能 value,从而生成一批候选口令。该过程存在明显的可并行性:对于同一个 PT,其最后一个 segment 的不同 value 之间没有依赖关系,因此可以将这些 value 划分给不同进程并行生成;同时,生成后的哈希计算也可以在不同进程或不同线程中独立完成。

因此,本次 MPI 实验需要解决的子问题包括:

- 如何在 MPI 多进程环境下实现口令猜测任务划分,使多个进程分工完成同一个口令猜测任务,而不是简单重复执行同一任务;
- 如何在保证优先队列逻辑正确的前提下,将单个 PT 的候选口令生成任务分配给多个 worker 进程;
- 如何减少 MPI 通信开销,避免大量字符串在进程间频繁传输;
- 如何比较串行版本与 MPI 版本在口令生成、哈希计算和训练阶段的时间差异,并分析 MPI 并行化带来的收益与开销。

本实验重点实现基础 MPI 并行版本,即采用 Master-Worker 结构:rank 0 作为主进程,负责训练模型、维护优先队列、分发任务与汇总结果;其他 rank 作为 worker 进程,负责接收任务并完成对应的口令生成或哈希计算。

2 多线程算法设计与实现

2.1 MPI 算法思路

本实验采用 Master-Worker 模式实现 MPI 并行,代码均存于此仓库的 lab4 中。整体思路如下:

1. 程序启动后调用 `MPI_Init` 初始化 MPI 环境,并通过 `MPI_Comm_rank` 和 `MPI_Comm_size` 获取当前进程编号和进程总数。

- rank 0 作为 master 进程，负责执行主控制逻辑，包括模型训练、优先队列初始化、PT 取出、任务分发、结果汇总和最终输出。
- rank 大于 0 的进程作为 worker 进程，不再重复执行完整的 main 逻辑，而是进入循环等待 master 发送任务。
- master 每次从优先队列中取出一个 PT，并确定该 PT 最后一个 segment 的候选 value 数量。
- master 将候选 value 按照 MPI 进程数量进行划分，不同 rank 负责不同区间的 value，从而实现单个 PT 内部的并行生成。
- worker 接收 master 发送的 prefix 和 value 区间后，在本地生成对应候选口令，并完成相应统计或哈希任务。
- 任务完成后，worker 先返回本轮生成数量，再将生成的候选口令返回 master，由 master 汇总到全局 guesses 列表中，更新全局生成数量、哈希时间等统计信息。
- 当生成口令数量达到实验设定上限后，master 向所有 worker 发送停止信号，所有进程调用 `MPI_Finalize` 退出。

该设计的核心思想是：rank 0 维护全局优先队列，保证 PT 生成顺序和概率逻辑正确；worker 只负责计算任务，不直接修改优先队列。因此，优先队列的状态不会出现并发修改问题，也避免了多进程重复维护不同版本队列造成的错误。

从任务划分角度看，本实验的 MPI 并行属于“单 PT 内部 value 划分”。对于一个给定 PT，最后一个 segment 中的不同 value 互相独立，因此可以自然地分配给多个进程处理。假设最后一个 segment 共有 n 个 value，MPI 进程数为 p ，则第 r 个进程负责的下标区间可以表示为：

$$start_r = \left\lfloor \frac{n \cdot r}{p} \right\rfloor, \quad end_r = \left\lfloor \frac{n \cdot (r + 1)}{p} \right\rfloor$$

该划分方式相比简单的固定 chunk 划分更均衡，可以较好地处理 n 不能被 p 整除的情况。

2.2 算法相关代码实现

本节主要展示 MPI 并行实现中的关键代码。完整代码可参考实验提交文件中的 `main.cpp` 和 `guessing.cpp`。

2.2.1 MPI 初始化与 rank 分工

在 `main.cpp` 中，程序首先初始化 MPI 环境，并根据 rank 区分 master 和 worker。rank 0 继续执行训练、队列初始化和任务分发逻辑，其他 rank 进入 worker 循环等待任务。

```
1 #ifdef MPI_PARALLEL
2 #include <mpi.h>
3 extern void MPI_WorkerLoop();
4 extern void MPI_StopWorkers();
5 #endif
```

```

6 ...
7 #ifdef MPI_PARALLEL
8     MPI_Init(&argc, &argv);
9     int mpi_rank = 0;
10    int mpi_size = 1;
11    MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
12    MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
13
14    // Master-Worker 模式:
15    // rank 0 继续执行 main 的训练、队列、hash、输出逻辑;
16    // 其他 rank 不再重复训练和初始化队列, 只进入 worker 循环等待任务。
17    if (mpi_rank != 0)
18    {
19        MPI_WorkerLoop();
20        MPI_Finalize();
21        return 0;
22    }
23 #else
24     int mpi_rank = 0;
25     int mpi_size = 1;
26 #endif
27 ...
28 #ifdef MPI_PARALLEL
29     MPI_StopWorkers();
30     MPI_Finalize();
31 #endif

```

2.2.2 Master-Worker 任务分发

在 `guessing.cpp` 中, master 根据当前 PT 最后一个 segment 的 value 总数, 将任务划分给不同 worker。每个 worker 接收任务后生成对应区间的候选口令。

```

1 static long long MPI_MasterGenerateValues(
2     vector<string> &guesses,
3     const string &prefix,
4     segment *seg_ptr,
5     int total)
6 {
7     int rank = 0;
8     int size = 1;
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
11
12    if (rank != 0)
13    {
14        return 0;
15    }
16
17    if (total <= 0)
18    {
19        return 0;
20    }

```

```

21
22 // 给 worker 发送任务
23 for (int dest = 1; dest < size; dest += 1)
24 {
25     int start = static_cast<int>((long long)total * dest / size);
26     int end = static_cast<int>((long long)total * (dest + 1) / size);
27     int count = end - start;
28
29     int header[2];
30     header[0] = static_cast<int>(prefix.size());
31     header[1] = count;
32
33     MPI_Send(header, 2, MPI_INT, dest, TAG_MPI_WORK, MPI_COMM_WORLD);
34
35     if (!prefix.empty())
36     {
37         MPI_Send(prefix.data(), header[0], MPI_CHAR, dest, TAG_MPI_PREFIX, MPI_COMM_WORLD);
38     }
39
40     for (int i = start; i < end; i += 1)
41     {
42         MPI_SendString(seg_ptr->ordered_values[i], dest, TAG_MPI_VALUE);
43     }
44 }
45
46 long long generated = 0;
47
48 // rank 0 自己负责第 0 块
49 int start0 = 0;
50 int end0 = static_cast<int>((long long)total / size);
51
52 guesses.reserve(guesses.size() + total);
53
54 for (int i = start0; i < end0; i += 1)
55 {
56     guesses.emplace_back(prefix + seg_ptr->ordered_values[i]);
57     generated += 1;
58 }
59
60 // 收回 worker 生成的 guesses
61 for (int src = 1; src < size; src += 1)
62 {
63     MPI_Status status;
64     long long recv_count = 0;
65
66     MPI_Recv(&recv_count, 1, MPI_LONG_LONG, src, TAG_MPI_RESULT_COUNT, MPI_COMM_WORLD, &status);
67
68     for (long long i = 0; i < recv_count; i += 1)
69     {
70         string guess = MPI_RecvString(src, TAG_MPI_RESULT_GUESS);
71         guesses.emplace_back(move(guess));
72     }
73
74     generated += recv_count;
75 }

```

```
76
77     return generated;
78 }
```

2.2.3 Worker 循环与停止信号

worker 进程在循环中等待 master 发送任务。当收到普通任务时进行口令生成；当收到停止信号时退出循环。

```
1 void MPI_WorkerLoop()
2 {
3     while (true)
4     {
5         MPI_Status status;
6         int header[2] = {0, 0};
7
8         MPI_Recv(header, 2, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
9
10        if (status.MPI_TAG == TAG_MPI_STOP)
11        {
12            break;
13        }
14
15        int prefix_len = header[0];
16        int value_count = header[1];
17
18        string prefix;
19        prefix.resize(prefix_len);
20
21        if (prefix_len > 0)
22        {
23            MPI_Recv(&prefix[0], prefix_len, MPI_CHAR, 0, TAG_MPI_PREFIX, MPI_COMM_WORLD, &status);
24        }
25
26        vector<string> local_guesses;
27        local_guesses.reserve(value_count);
28
29        for (int i = 0; i < value_count; i += 1)
30        {
31            string value = MPI_RecvString(0, TAG_MPI_VALUE);
32            local_guesses.emplace_back(prefix + value);
33        }
34
35        long long result_count = static_cast<long long>(local_guesses.size());
36        MPI_Send(&result_count, 1, MPI_LONG_LONG, 0, TAG_MPI_RESULT_COUNT, MPI_COMM_WORLD);
37
38        for (string &guess : local_guesses)
39        {
40            MPI_SendString(guess, 0, TAG_MPI_RESULT_GUESS);
41        }
42    }
```

```

43 }

1 void MPI_StopWorkers()
2 {
3     int rank = 0;
4     int size = 1;
5     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6     MPI_Comm_size(MPI_COMM_WORLD, &size);
7
8     if (rank != 0)
9     {
10         return;
11     }
12
13     int header[2] = {0, 0};
14
15     for (int dest = 1; dest < size; dest += 1)
16     {
17         MPI_Send(header, 2, MPI_INT, dest, TAG_MPI_STOP, MPI_COMM_WORLD);
18     }
19 }

```

2.2.4 口令生成与 SIMD 哈希结合

本实验在 MPI 任务划分基础上保留了已有的 SIMD 哈希实现。候选口令生成后，程序以 4 个口令为一组调用 SIMD 版本的 MD5 哈希函数，从而在进程间 MPI 并行的基础上进一步利用数据级并行。

```

1 //单 segment
2 #ifdef MPI_PARALLEL
3 long long generated = MPI_MasterGenerateValues(guesses, "", a, pt.max_indices[0]);
4 total_guesses += generated;
5 #endif
6 ...
7 //多 segment
8 #ifdef MPI_PARALLEL
9 int last_idx = pt.content.size() - 1;
10 long long generated = MPI_MasterGenerateValues(guesses, guess, a, pt.max_indices[last_idx]);
11 total_guesses += generated;
12 #endif

```

3 实验及结果分析

3.1 实验结果

实验在相同数据集和相同口令生成规模下，分别测试串行版本与 MPI 版本的性能。实验中生成候选口令数量约为一千万级别，哈希部分使用 SIMD 优化。

表 1: 串行版本与 MPI 版本实验结果对比

指标	串行版本	MPI 版本
Guess time / s	0.404968	0.381680
SIMD Hash time / s	1.756640	1.953420
Train time / s	31.310400	27.012200
Total guesses generated	10,035,780	10,187,204

为了更直观地观察 MPI 并行化对各阶段的影响，对主要时间指标计算相对变化如下：

表 2: MPI 版本相对串行版本的性能变化

指标	计算方式	结果
Guess 阶段加速比	$0.404968/0.381680$	约 1.061
Hash 阶段加速比	$1.756640/1.953420$	约 0.899
Train 阶段时间比	$31.310400/27.012200$	约 1.159
生成数量差异	$(10,187,204 - 10,035,780)/10,035,780$	约 1.51%

实验运行截图如下：

```
Guess time:0.404968seconds
SIMD Hash time:1.75664seconds
Train time:31.3104seconds
Total guesses generated: 10035780
```

图 1: 串行版本运行结果截图

```
MPI Guess time:0.38168seconds
SIMD Hash time:1.95342seconds
Train time:27.0122seconds
Total guesses generated: 10187204
```

图 2: MPI 版本运行结果截图

3.2 结果分析

从实验结果可以看出，MPI 版本在口令生成阶段相较串行版本有一定加速。串行版本的 Guess time 为 0.404968 秒，MPI 版本的 Guess time 为 0.381680 秒，对应加速比约为 1.061。这说明将单个 PT 最后一个 segment 的 value 划分给多个 MPI 进程后，确实能够降低候选口令生成阶段的时间。

然而，MPI 版本的 SIMD Hash time 为 1.953420 秒，略高于串行版本的 1.756640 秒，说明在当前实现中，哈希阶段并没有获得明显加速，反而受到一定额外开销影响。可能原因包括：

- MPI 任务分发与结果收集引入了额外通信开销；
- rank 0 仍承担较多控制逻辑，包括优先队列维护、任务划分和输出统计；
- 当前实现主要针对单个 PT 内部 value 划分，若不同 PT 之间 value 数量差异较大，可能出现 worker 负载不均衡；
- 哈希部分虽然使用 SIMD，但 MPI 进程之间的同步与通信可能抵消部分并行收益；
- 实验生成数量并非完全一致，MPI 版本生成口令数量略多，导致 Hash time 比较时存在一定偏差。

训练阶段方面，MPI 版本测得训练时间为 27.012200 秒，低于串行版本的 31.310400 秒。但训练阶段本身不是本实验 MPI 并行优化的核心部分，该差异可能与实验运行时系统负载、I/O 状态和调度环境有关。因此，训练时间不应作为判断 MPI 优化效果的主要依据，更应重点比较 Guess time 和 Hash time。

当前版本已经满足基础 MPI 并行要求：多个进程不是简单重复执行完整任务，而是在 rank 0 控制下分工完成口令猜测过程。rank 0 负责优先队列和任务调度，worker 负责接收 master 分发的 value 并生成局部候选口令，随后将生成结果返回 rank 0，由 rank 0 继续进行统一哈希和统计，整体符合 Master-Worker 模式。

不过，当前实现仍然存在进一步优化空间。首先，当前版本每次只从优先队列中取出一个 PT，再对该 PT 内部的 value 进行划分。这属于“单 PT 内部并行”。如果某个 PT 的候选 value 数量较少，则 worker 进程可能得不到足够任务，导致并行效率下降。其次，口令生成和哈希计算在整体流程上仍然偏批处理模式，即生成一批后再进行哈希，尚未形成真正的生成-哈希流水线。

后续可以从以下方向继续优化：

- 实现 PT 层面的并行：rank 0 一次从优先队列中取出多个 PT，将不同 PT 分发给不同 worker 并行生成，等待本轮任务完成后再统一将新 PT 放回优先队列；
- 实现生成-哈希流水线：将部分进程用于口令生成，部分进程用于哈希计算，使生成和哈希两个阶段重叠执行；
- 使用非阻塞通信：通过 MPI_Isend 和 MPI_Irecv 减少同步等待；
- 改进负载均衡策略：根据每个 PT 的候选 value 数量动态分配任务，而不是简单按 rank 固定划分；
- 将 MPI 与 OpenMP/Pthread 结合，实现进程间 MPI 并行和进程内多线程并行的混合优化。

综上，本实验实现了基于 MPI 的口令猜测并行化，并完成了串行版本与 MPI 版本的性能对比。实验结果表明，MPI 并行能够在口令生成阶段带来一定加速，但由于通信、同步和负载均衡问题，整体性能仍有进一步优化空间。